

```
temp.py x
1 import os
2 import openai
3 import secret
4 openai.api_key=secret.api_key
5
6 # WRITE YOUR CODE HERE
7 models = openai.Model.list()
8 print(models['data'][:2])
```

Collapse

< > ⌕ ☰

Models

Endpoint Signatures

Each endpoint has a signature. This means either a `GET` or `POST` request followed by the URI. Below is an example of the Completions endpoint signature. It uses a `POST` method. Other signatures may require a parameter. These will be identified by curly brackets `{ }`.

Completions Endpoint Signature

POST https://api.openai.com/v1/completions

Endpoint signatures are provided in the examples below to give you a better idea of how the OpenAI is doing behind the scenes.

List Model Endpoint

List Model Endpoint Signature

GET https://api.openai.com/v1/models

This code prints out a long list of all of the available models and accompanying metadata. You can see why this is sometimes referred to as the metadata endpoint.

```
models = openai.Model.list()
print(models)
```

TRY IT!

[<Model model id=text-search-babbage-doc-001 at 0x7f7a3276d8b0> JSON: { "created": 1651172509, "id": "text-search-babbage-doc-001", "object": "model", "owned_by": "openai-dev" }, <Model model id=gpt-3.5-turbo-16k at 0x7f7a3278dbd0> JSON: { "created": 1683758102, "id": "gpt-3.5-turbo-16k", "object": "model", "owned_by": "openai-internal" }]

In a previous discussion, we talked about the four different types of models: `davinci`, `curie`, `babbage`, and `ada`. Each of these models varies in terms of capabilities, speed, and cost. However, the list of available models is greater than four. We will see later how we can use models like `text-search-babbage-query-001`.

Printing all of the models looks like it may be a JSON object or a dictionary. In reality, the return type from the API call is `OpenAIObject`. However, we can treat it like a dictionary. The code below prints out the first model and its metadata.

```
models = openai.Model.list()
print(models["data"][0])
```

TRY IT!

Try this variation:

- Create a list of all the model IDs without any other metadata.

models = openai.Model.list()
model_ids = [model["id"] for model in models["data"]]
print(model_ids)

TRY IT!

{ "created": 1651172509, "id": "text-search-babbage-doc-001", "object": "model", "owned_by": "openai-dev" }

Retrieve Model Endpoint

List Engines Endpoint Signature

GET https://api.openai.com/v1/models/{model}

Instead of parsing the list of models, you can retrieve metadata about a specific one if you know the `id` for the model. The code sample below prints out the metadata for the `text-davinci-002` model.

```
model = openai.Model.retrieve("text-davinci-002")
print(model)
```

TRY IT!

{ "created": 1649880484, "id": "text-davinci-002", "object": "model", "owned_by": "openai" }

Select the code snippet that prints the total number of models.

☒ models = openai.Model.list()
print(len(models["data"]))

☐ models = openai.Model.list()
print(len(models))

☐ models = openai.Model.retrieve("text-davinci-002")
print(len(models["data"]))

The correct answer is:

models = openai.Model.list()
print(len(models["data"]))

Since we want to count all of the models, we need to use `openai.Model.list()`. Using the retrieve method will only return a single model. In addition, all of the model metadata is stored in a list associated with the `"data"` key. We should calculate the length of `models["data"]`.

Check It

Next